Computer Networks

Chapter 4 - Network Layer: Data Plane

Lecture-wise study notes | Lectures 13-16

Based on the four lecture PDFs dated 1, 6, 8 and 13 October 2026. The Chapter 3 section of Lecture 13 is excluded. Written for a beginner: terms are introduced before they are used, and processes are explained as a sequence of events.

How to study these notes

Read one lecture at a time. First understand the explanation and process tables; then cover the worked answers and try the calculations yourself. Finish with the formula sheet at the end of that lecture. Repeated animation slides are combined into one explanation.

Lecture	Main topics in the supplied material
13 - notes pp. 2-5	Network-layer overview; data/control planes; service models; Internet Protocol; IPv4 header and addressing
14 - notes pp. 6-11	Classful/classless addressing; special/private addresses; subnets; masks; FLSM and VLSM
15 - notes pp. 12-17	DHCP; hierarchical allocation; aggregation; longest-prefix matching; IPv4 fragmentation and MTU
16 - notes pp. 18-26	NAT/PAT; IPv6 and transition; generalized forwarding/OpenFlow; middleboxes; Internet architecture

Source boundaries - read once

The notes follow the slides' topic headings within each lecture. The chapter roadmap also lists router switching, buffering and scheduling, but the uploaded PDFs contain no dedicated teaching slides on those mechanisms. Their roadmap names are explained briefly; missing lessons are not invented.

The FLSM, VLSM and route-summarization videos mentioned in the slides were not supplied. Added practice is marked **Teaching example**, so it cannot be mistaken for a reconstruction of those videos. Some image-only slide details were not recoverable; the final source audit identifies them. These notes therefore should not be treated as a verified replacement for those unreadable details or the missing videos.

Diagrams: all illustrations here are text-based process tables, field layouts and route descriptions. No slide pictures are reproduced. The sequences explain the visible concepts; they do not claim to recover hidden PowerPoint animation timing.

Attribution: course slides by Nauman Moazzam Hayat, CS 3001, National University of Computer & Emerging Sciences; adapted from Jim Kurose and Keith Ross, *Computer Networking: A Top-Down Approach*, 8th edition, Pearson, 2020. Original slide copyright remains with its authors.

Lecture 13 | 1 October 2026

Network layer: overview

Source: Lecture 13, PDF pp. 22-29 and 35. Chapter 3 material on PDF pp. 1-21 is excluded.

Start here: what is travelling through a network?

A **host** is an end device, such as a laptop or server. A **link** is a connection over which nearby devices communicate. A **router** connects IP networks and sends packets toward their destinations. A **protocol** is an agreed set of message formats and communication rules.

A large message is carried in smaller units. Different layers wrap that information in their own control information. A **header** contains instructions such as addresses; a **payload** is the information being carried inside that particular wrapper.

Layer	Unit and its contents	Job in this chapter's story
Application	Application message	The information the program wants to exchange
Transport	TCP/UDP header + application data	Provides communication for applications; its unit is often called a segment (UDP also uses the term datagram)
Network	IP header + transport unit	The IP packet, also called an IP datagram, travels from sending host to receiving host
Link	Link header + IP packet + possible trailer	A frame carries the packet across one link; a trailer is control information placed at the end

Encapsulation means putting one unit inside another. On receipt, **decapsulation** removes the wrappers. Payload is relative: an IP packet is the link frame's payload, while a TCP segment is the IP packet's payload.

Two key network-layer functions

Forwarding is one router's local decision: which output interface should this arriving packet use? A **forwarding table** holds the rules for that choice. **Routing** determines paths through the network and helps build those tables. Planning an entire road trip resembles routing; choosing the correct exit at one junction resembles forwarding.

Step	Packet's journey
1. Sender	Encapsulates the transport segment in an IP datagram and passes it to the link layer.
2. Each router	Receives a frame, examines the IP header, consults its forwarding information and moves the packet from an input to an output interface.
3. Next link	Carries the packet in a link-layer frame suitable for that link.
4. Receiver	Passes the IP payload to the transport protocol identified in the IP header.

An **interface** is a device's connection to a link. An input/output **port** on a router means such a connection; it is different from a TCP/UDP port number that identifies an application's communication endpoint.

Lecture 13 | Control and service models

Network layer: data plane, control plane

Source: Lecture 13, PDF pp. 26-36.

The **data plane** performs forwarding for individual packets at a router. The **control plane** determines the forwarding information using a wider view of the network.

Approach	How the forwarding information is produced	What happens when a packet arrives?
Per-router control	Each router runs a routing algorithm. Routers exchange information and compute their own forwarding tables.	The router's data plane uses its local table.
Software-Defined Networking (SDN)	A logically centralized controller computes and installs rules in devices. The slide's CA means control agent: the component communicating with the controller.	The device applies the installed rules locally. It need not ask the controller about every packet.

Algorithm means a procedure for solving a problem. In SDN, logically centralized describes the organization of control; it need not mean one unreplicated physical machine.

Network service model

A **service model** describes what a network promises. A **flow** is a related stream of packets. **Bandwidth** is a data-carrying rate, often measured in bits per second. **Delay** is waiting/travel time; **jitter** is variation in packet delay or arrival spacing. **QoS** means Quality of Service.

The slides distinguish promises about an individual datagram, such as delivery or delivery within 40 ms, from promises about a flow, such as ordering, minimum bandwidth or controlled jitter.

Model in the slides	Bandwidth promise	Delivery/loss guarantee?	Order?	Timing?
Internet best effort	None	No	No	No
ATM Constant Bit Rate	Constant rate	Yes	Yes	Yes
ATM Available Bit Rate	Guaranteed minimum	No	Yes	No
Internet Intserv Guaranteed	Yes	Yes	Yes	Yes
Internet Diffserv	Possible	Possibly	Possibly	No

ATM is a network technology; Intserv reserves service for flows; Diffserv differentiates traffic classes. These rows compare models, not guarantees automatically available on every Internet connection.

Best effort means the Internet attempts delivery without promising success, a delivery time, an order or a minimum rate. It succeeded because simple mechanisms scale well, enough capacity often gives acceptable real-time performance, nearby replicated servers reduce distance, and congestion-controlled applications adjust their sending rates. A CDN is a Content Delivery Network: distributed servers that bring content closer to users. Elastic traffic can tolerate a changing transfer rate.

IP: the Internet Protocol

IP is connectionless: it does not first establish an end-to-end network-layer connection. It maintains no per-connection end-to-end state and does not itself recover lost packets or guarantee ordering. TCP can provide reliability above IP; applications using UDP must arrange any extra service they need. The slide's broad statement about no congestion indicators needs qualification: its own header diagram includes ECN congestion marking.

The Internet network layer includes IP, path-selection mechanisms (the slides name OSPF, BGP and SDN), and control/error reporting such as ICMP. ICMP carries control/error messages; IGMP, also mentioned, concerns group membership for multicast, not general error recovery.

Lecture 13 | IPv4 datagram format

IPv4 header: understand the fields

Source: Lecture 13, PDF pp. 37-39. Field-width clarification: RFC 791.

IPv4 means Internet Protocol version 4. Its usual header is **20 bytes**; options can extend it to **60 bytes**. A bit is a binary digit, 0 or 1. Eight bits form one byte. The following table replaces the header picture while retaining the field order. Each row of the normal layout occupies 32 bits.

Header row	Fields in left-to-right order
1	Version (4 bits); IHL (4); service field (8); Total Length (16)
2	Identification (16); Flags (3); Fragment Offset (13)
3	TTL (8); Protocol (8); Header Checksum (16)
4	Source IPv4 address (32)
5	Destination IPv4 address (32)
Optional area	Options and padding, if present; then the payload follows

Field	Beginner explanation
Version	Says how to interpret the header. For IPv4 it is decimal 4, binary 0100.
IHL: Internet Header Length	Says where the header ends and payload begins. Its value counts 4-byte words, not individual bytes. Actual header length = IHL x 4 bytes.
Service field	The slide divides its 8 bits into 6 bits for differentiated services and 2 bits for ECN. The first can identify a traffic-treatment class; ECN can signal congestion. Neither automatically guarantees QoS.
Total Length	Entire IPv4 datagram size in bytes: header plus payload. A 16-bit field can represent at most 65,535 bytes, approximately 64 KB.
Identification	Helps associate fragments with the same original datagram. Used with other identifying fields during reassembly.
Flags	Includes DF (Don't Fragment) and MF (More Fragments). Their use is explained in Lecture 15.
Fragment Offset	Locates a fragment's data within the original payload, in units of 8 bytes.
TTL: Time To Live	A hop-limit safeguard. Routers decrement it; a packet cannot circulate forever. A hop is one forwarding step through a router.
Protocol	Identifies what IP should deliver its payload to, such as TCP or UDP. It is not an application port number.
Header Checksum	Detects certain errors in the IPv4 header. It does not check the whole payload or repair corruption. Because TTL changes, forwarding affects this checksum.
Source / destination	The sender's and intended receiver's IPv4 interface addresses, each 32 bits.
Options	Additional information, such as a timestamp or route recording. Padding keeps header length on a 4-byte boundary.

Do not confuse the lengths: IHL describes the header alone; Total Length includes the IP payload. Likewise, the IP payload can contain its own transport header; it is not necessarily pure application data.

Lecture 13 | Examples and formula sheet

IPv4 header examples

Source: Lecture 13, PDF pp. 37-41.

Slide example 1: IHL = 0101. Binary 0101 equals 5. Each IHL unit represents 4 bytes, so the header is 5 x 4 = **20 bytes**. This is the minimum header and has no options.

Slide example 2: IHL = 1010. Binary 1010 equals 10. The header is 10 x 4 = **40 bytes**. Compared with the normal 20-byte header, the extra 20 bytes belong to the options/padding area. Do not interpret IHL = 10 as a 10-byte header.

Slide overhead example. With normal headers, TCP adds 20 bytes and IP adds 20 bytes. Their combined overhead is **40 bytes** before counting application-layer overhead or link-layer framing. Overhead means supporting information transmitted in addition to the useful data being discussed.

Default TTL values

The lecture lists 64 for Linux/Mac OS X and 128 for Windows. Treat these as lecture examples of initial defaults, not universal identifiers. A received packet has already lost TTL at routers, and a sender can configure its initial value. TTL alone therefore provides a clue about an operating system, not proof.

IPv4 addressing: introduction

An IPv4 address identifies an **interface**, rather than a whole multi-interface router. A router usually has multiple interfaces and thus multiple addresses. A host may have Ethernet and Wi-Fi interfaces. **Dotted-decimal notation** writes the 32 bits as four 8-bit numbers, called octets, separated by dots. Each octet ranges from 0 to 255.

Slide address	First octet	Second	Third	Fourth
Decimal	223	1	1	1
Binary	11011111	00000001	00000001	00000001

For 223: 128 + 64 + 16 + 8 + 4 + 2 + 1 = 223. Reading the binary weights 128, 64, 32, 16, 8, 4, 2, 1 gives 11011111. The complete IPv4 space has 2^32 = 4,294,967,296 bit patterns. This is not a count of assignable public host addresses: some ranges have special uses, and private addresses can be reused.

Lecture 13 - all calculation rules

Formula / rule	Meaning and units
1 byte = 8 bits	Convert storage/header sizes consistently.
H = 4 x IHL	H is IPv4 header length in bytes; valid ordinary IHL values are 5-15, giving 20-60 bytes.
IP payload = Total Length - H	Subtract only the IP header to obtain IP payload bytes.
Normal TCP + IPv4 overhead = 20 + 20 = 40 bytes	Assumes neither header has options; excludes link/application overhead.
IPv4 address space = 2^32	Number of possible 32-bit address patterns.
Largest Total Length = 2^16 - 1 = 65,535 bytes	IPv4 header and payload together.
TTL after k forwarded hops = initial TTL - k	Applies while the packet remains forwardable; reaching the limit causes discard.

Self-check: forwarding is the local action; routing produces path information. Best effort promises no particular outcome. Header size, payload size and total size are three different quantities.

Lecture 14 | 6 October 2026

Classful IPv4 addressing

Source: Lecture 14, PDF pp. 3-7. The class layout below is explanatory background; the image-only slide details could not be verified individually.

Recall that an IP address belongs to an interface. Its **network part** identifies a network; its **host part** distinguishes interfaces within that network. Classful addressing used fixed boundaries decided by the leading bits of an address. **Unicast** means sending toward one recipient; **multicast** means sending toward members of a group.

Class	Leading bits	First-octet range	Default network/host split	Purpose
A	0	0-127 as a bit-pattern range	/8: 8 network, 24 host bits	Unicast, with special ranges excluded
B	10	128-191	/16: 16 network, 16 host bits	Unicast
C	110	192-223	/24: 24 network, 8 host bits	Unicast
D	1110	224-239	No ordinary network/host split	Multicast
E	1111	240-255	No ordinary network/host split	Reserved/special uses

Ordinary historical Class A network numbers are usually given as 1-126: 0 and 127 have special meanings. In classless addressing, a /prefix must be supplied; you cannot infer today's actual subnet mask merely from this class table.

Why does the lecture say classful addressing wastes space?

An organization needing slightly more than 256 total addresses could not be given an arbitrary intermediate classful block. A Class B block has 65,536 addresses, potentially leaving most unused. Classless addressing allows a more suitable power-of-two-sized block.

Class	Theoretical block-count calculation in slide	Total addresses per block
A	2^7 = 128 bit-pattern network choices	2^24 = 16,777,216
B	2^14 = 16,384	2^16 = 65,536
C	2^21 = 2,097,152	2^8 = 256
D / E	Slide shows one class-wide space for each	2^28 = 268,435,456 patterns in each space

The fixed class-identifying bits do not vary when counting classful network numbers. Thus Class B has 16 network bits but only 14 variable network-number bits. The slide calls block sizes numbers of nodes; technically they are **total address counts**. Ordinary usable-host counts exclude the network and broadcast addresses.

Finding the class - teaching examples

75.3.1.28: first octet 75 falls in the Class A range; the old default mask is 255.0.0.0. **200.23.16.0:** 200 falls in Class C; the classful default would be /24. However, 200.23.16.0/23 explicitly says /23 and must be interpreted classlessly.

Exam habit: first identify whether the question asks for an old class or supplies a prefix. These are different tasks.

Lecture 14 | Special and private addresses

Special IP addresses (Classful)

Source: Lecture 14, PDF pp. 8-9. Standard meanings clarified using RFC 1122 and RFC 1918; some original image labels were not readable.

Use the slide's example **75.3.1.28** under its Class A default /8. Here 75 is the network part, and 3.1.28 is the host part.

Address / pattern	Meaning in this example
75.0.0.0	Network address: keep the network bits and set all host bits to 0. Identifies the network, not an ordinary host.
75.255.255.255	Directed broadcast: keep network bits and set all host bits to 1. Identifies all hosts on that destination network conceptually; forwarding such broadcasts is a separate policy decision.
255.255.255.255	Limited broadcast to the local network. Routers do not carry it onward as an ordinary routed broadcast.
0.0.0.0	Unspecified source address in contexts such as a host initially obtaining an address. It is not a normal assigned interface address.
0.3.1.28	Historical classful notation for a specified host on this network. Do not configure this as a normal modern host address.
127.0.0.0/8; commonly 127.0.0.1	Loopback: communication remains inside the host's protocol stack. It does not go out onto the physical link.

The last two rows explain historical special-address concepts; they are not alternative addresses a normal Internet peer can use to contact your laptop.

Private IP addressing

A **private address** is intended for use inside private networks and is not globally unique. Different homes can both use 192.168.1.10 because those are separate local spaces. A public address is used for globally routed communication. Lecture 16 explains how NAT connects private-addressed devices to outside services.

Private block	Full range	Number of address bits left after prefix
10.0.0.0/8	10.0.0.0 - 10.255.255.255	24
172.16.0.0/12	172.16.0.0 - 172.31.255.255	20
192.168.0.0/16	192.168.0.0 - 192.168.255.255	16

Only 172.16 through 172.31 is private within the 172 first-octet space. **172.40.1.1 is not in that private block.** Likewise, the private 192 block is specifically 192.168/16, not every address beginning 192.

The slides associate these ranges with commercial/residential use. Those labels are examples of common deployment, not rules restricting who may use each block. A home or organization may choose any suitable RFC 1918 space.

Network address versus broadcast: the story

The prefix tells you which bits belong to the subnet. To name the subnet itself, clear its host bits to 0. To address every host in that subnet using its broadcast address, set those host bits to 1. The same procedure works with classless prefixes; it is not limited to whole octets.

Lecture 14 | CIDR and subnets

IP addressing: CIDR

Source: Lecture 14, PDF pp. 10-15.

CIDR means Classless Inter-Domain Routing, pronounced cider. It uses an explicit prefix length instead of Class A/B/C boundaries. In **a.b.c.d/p**, the first p bits form the network prefix and the remaining 32 - p bits form the host part. A longer prefix leaves fewer host bits and therefore describes a smaller block.

Slide example: 200.23.16.0/23

1. The prefix occupies 23 bits: the first two octets (16 bits) plus 7 bits of the third octet.

2. Host bits = 32 - 23 = **9**. Total addresses = 2^9 = **512**.

3. Its subnet mask is **255.255.254.0**: 23 leading 1s, followed by 9 zeros.

4. The third octet advances in blocks of 2. This block covers **200.23.16.0 through 200.23.17.255**.

5. For an ordinary subnet, network = 200.23.16.0, broadcast = 200.23.17.255, usual host range = 200.23.16.1 through 200.23.17.254, giving **510** usable host addresses.

Subnets

A **subnet** is a set of interfaces that can communicate across the local link structure without going through a router. Switches or Wi-Fi equipment can connect those interfaces. Devices in an ordinary IP subnet share the same prefix. High-order bits means the leftmost bits; low-order bits means the rightmost bits.

The three-subnet diagram, retold

Local island	Addresses visible in slide	Router's interface on that island
223.1.1.0/24	223.1.1.1, .2 and .3	223.1.1.4
223.1.2.0/24	223.1.2.1 and .2	223.1.2.9
223.1.3.0/24	223.1.3.1 and .2	223.1.3.27

To send from 223.1.1.1 to 223.1.2.2, the sender recognizes that the destination is in another subnet and sends toward its router. The router receives on the .1 subnet's interface and forwards toward the .2 subnet. The two interfaces belong to the same router, but to different subnets.

The island rule

Imagine disconnecting each interface from the inside of its host/router while leaving the external links and switches in place. Each connected island of interfaces is a subnet. Count the networks, not simply the routers. The more complex slide has **six /24 subnets**: 223.1.1.0, 223.1.2.0, 223.1.3.0, 223.1.7.0, 223.1.8.0 and 223.1.9.0. Links between routers also form subnets.

Slide caution: the topology labels some inter-router interfaces with a final .0 while also naming /24 subnets. Under ordinary /24 rules, .0 identifies the subnet, not an assignable host. Use the diagram to identify its six islands; do not copy those inconsistent labels into a real configuration.

Lecture 14 | Subnet mask and FLSM

Subnetting; Subnet Mask (SM)

Source: Lecture 14, PDF pp. 15-17 and 19-20.

Subnetting divides an allocated block into smaller internal networks. Outsiders may use one parent route while internal routers use more detailed subnet routes. This works when the parent is advertised as an aggregate; subnetting does not force every Internet router to learn every internal subnet.

A **subnet mask** is a 32-bit pattern, not a destination address. Its leading 1s select the prefix; its trailing 0s select the host part. **Bitwise AND** compares corresponding bits: only 1 AND 1 produces 1; 1 AND 0, 0 AND 1 and 0 AND 0 produce 0. Therefore address AND mask keeps network bits and clears host bits.

Default classful mask	Equivalent prefix
255.0.0.0	/8
255.255.0.0	/16
255.255.255.0	/24

Worked expansion of the slide: 128.125.0.0 into 8 equal subnets

The slides state the original Class B mask is /16 and the internal result is 255.255.224.0. Here is the calculation behind that result. This is FLSM: **Fixed Length Subnet Mask**, meaning equal-size subnets use the same prefix length.

1. Start with **128.125.0.0/16**. There are 16 host bits available before subdivision.

2. Borrow b bits so that 2^b gives at least 8 subnets. Since 2^3 = 8, borrow **3 bits**.

3. New prefix = 16 + 3 = **/19**. The third mask octet is 11100000 = 128 + 64 + 32 = **224**, so the mask is **255.255.224.0**.

4. Host bits per subnet = 32 - 19 = **13**. Each has 2^13 = **8,192 addresses**, usually 8,190 usable hosts.

5. The changing third-octet block size is 256 - 224 = **32**. The subnet starts are therefore 0, 32, 64, 96, 128, 160, 192 and 224 in that octet.

Subnet prefixes	Address range of representative subnets
128.125.0.0/19; 128.125.32.0/19	First: 128.125.0.0 - 128.125.31.255
128.125.64.0/19; 128.125.96.0/19	Second: 128.125.32.0 - 128.125.63.255
128.125.128.0/19; 128.125.160.0/19	Continue by adding 32 to the third octet.
128.125.192.0/19; 128.125.224.0/19	Last: 128.125.224.0 - 128.125.255.255

How the internal router recognizes a subnet - teaching example

Destination **128.125.70.9**, mask /19: third octet 70 = 01000110; AND with 224 = 11100000 gives 01000000 = **64**. The final mask octet is 0, so the final result is **128.125.64.0/19**. The router uses the matching subnet entry's output interface. If several prefixes match, it must choose the longest matching prefix, as Lecture 15 explains.

The parent and first subnet share the written address 128.125.0.0, but **/16 and /19 identify different-sized blocks**.

Lecture 14 | Classless examples and VLSM

Difference between Classful, CIDR, FLSM and VLSM

Source: Lecture 14, PDF pp. 18-20. CIDR terminology clarified using RFC 4632.

Term	Question it answers
Classful addressing	Does the old A/B/C class determine a fixed block boundary?
CIDR / classless addressing	Is the block described by an explicit, flexible prefix rather than an old class boundary?
FLSM	When subdividing this block, do all resulting subnets have equal size?
VLSM	Can resulting subnets have different prefix lengths and sizes?

VLSM means Variable Length Subnet Mask. It fits different subnet sizes to different needs. The slides distinguish allocation to organizations from internal allocation by an administrator. That is useful context, but CIDR is not simply VLSM performed by ICANN. Registries and providers allocate prefixes hierarchically; administrators can subnet their allocation using FLSM or VLSM.

Teaching example: allocate 100, 50 and 20 hosts from 192.168.10.0/24

This added example teaches the method; it does not reproduce the unavailable classroom VLSM video or the image-only numerical on PDF p. 18.

1. Sort needs from largest to smallest: **100, 50, 20**. This makes it easier to keep large blocks aligned and avoid overlap.

2. Allow for a network and broadcast address. For 100 hosts, find h with 2^h - 2 >= 100. Six host bits give 62, too few; seven give 126. Choose **/25**, a 128-address block.

3. For 50 hosts, six host bits give 62: choose **/26**, a 64-address block.

4. For 20 hosts, five host bits give 30: choose **/27**, a 32-address block.

5. Place each block at the next valid boundary in the remaining space.

Need	Subnet; mask	Usable range	Broadcast
100	192.168.10.0/25; 255.255.255.128	.1 through .126	192.168.10.127
50	192.168.10.128/26; 255.255.255.192	.129 through .190	192.168.10.191
20	192.168.10.192/27; 255.255.255.224	.193 through .222	192.168.10.223

In the usable-range column the shared first three octets are 192.168.10. The remaining block, **192.168.10.224/27**, is unused. Check: 128 + 64 + 32 + 32 = 256 addresses, exactly the parent /24. Every allocation fits, starts at a valid boundary and avoids its neighbors.

Why not start the /26 at .100? A /26 is aligned on multiples of 64 in the final octet: 0, 64, 128 or 192. An arbitrary address does not become a subnet boundary just because you write /26 beside it.

The lecture calls FLSM/VLSM very important and directs students to two separate videos, FLSM first. Their exact numerical questions and solutions remain outside the supplied files.

Lecture 14 | Formula sheet

All addressing and subnetting calculation rules

These rules collect the lecture's calculations and the derivations used in the teaching examples. Assume ordinary IPv4 broadcast-capable subnets unless the question says otherwise.

Formula / procedure	How to use it
h = 32 - p	p = prefix length; h = number of host bits.
Total addresses = 2^h	Counts the entire subnet, including network and broadcast.
Usual usable hosts = 2^h - 2	Excludes network and broadcast. Do not apply blindly to special /31 or /32 uses.
Network address = IP AND mask	Keep prefix bits; clear host bits.
Broadcast = network with all host bits set to 1	Last address in the ordinary subnet.
First usable = network + 1	Next address after the network address.
Last usable = broadcast - 1	Address immediately before broadcast.
b = smallest integer with 2^b >= S	Bits to borrow for at least S equal subnets; all resulting subnets count.
New prefix = old prefix + b	FLSM subdivision.
Number of equal subnets = 2^b	Each borrowed bit doubles the number of subnets.
h = smallest integer with 2^h - 2 >= required hosts	Choose an ordinary subnet size from host demand.
Prefix for that demand = 32 - h	Converts the required host bits to a prefix.
Octet increment = 256 - mask octet	Use in the octet where the boundary changes; not blindly in an octet equal to 255 or 0.

Mask-building aid

An octet has weights 128, 64, 32, 16, 8, 4, 2, 1. Consecutive leading 1s produce the following partial mask octets. Fill earlier mask octets with 255 and later ones with 0.

Leading 1 bits in partial octet	Mask value	Block increment in that octet
1	128	128
2	192	64
3	224	32
4	240	16
5	248	8
6	252	4
7	254	2
8	255	Boundary completes this octet

Example: /19 contains 16 full prefix bits plus 3 more, so the mask is 255.255.224.0. /27 contains 24 full prefix bits plus 3 more, so it is 255.255.255.224. The same 224 appears in different octets because the prefix boundary is different.

Subnetting answer checklist

Give the prefix and mask, network address, broadcast address, usable range and host capacity. Check alignment and overlap. State whether the question wants total addresses or usable hosts. In VLSM, allocate the largest requirement first and check every subnet still fits inside the parent.

Concept recap: a class is a historical address category; a prefix tells you the actual network boundary. A subnet is a network, not a router. A mask extracts a subnet ID; it does not by itself uniquely identify that subnet.

Lecture 15 | 8 October 2026

IP addresses: how to get one? DHCP

Source: Lecture 15, PDF pp. 3-8.

There are three separate allocation questions: how a host gets an address inside its network; how an organization gets a block from an ISP; and how an ISP gets its block. An **ISP** is an Internet Service Provider. Start with the host.

A host can be configured manually by an administrator or automatically using **DHCP: Dynamic Host Configuration Protocol**. DHCP is an application-layer protocol that helps configure network-layer settings. It uses UDP at the transport layer. The server uses UDP port **67**; the client uses port **68**.

A **lease** is permission to use an address for a limited time. A client can renew it. Leasing allows addresses to be reused when devices leave and makes joining a network convenient for mobile users. Plug-and-play means settings can be learned without a person entering them each time.

DHCP client-server scenario - the four-message story

The slide's server is **223.1.2.5**, offering **223.1.2.4** for **3,600 seconds**. A broadcast reaches the local network rather than one already-known individual recipient.

Message	Who speaks?	Meaning and visible addressing
Discover	Client	I need configuration; is there a DHCP server? Source 0.0.0.0:68, destination 255.255.255.255:67.
Offer	Server	You may use 223.1.2.4 for the proposed lease. Source 223.1.2.5:67; the slide broadcasts to client port 68.
Request	Client	I choose/request that configuration. The slide uses source 0.0.0.0:68 and broadcasts to server port 67.
ACK	Server	The allocation is confirmed. The slide broadcasts the reply to client port 68.

Remember **DORA**: Discover, Offer, Request, Acknowledge. The offered-address field, labeled **yiaddr**, means your IP address in the slide. This is information inside DHCP; it is not necessarily the outer IP destination.

The diagram shows transaction ID 654 for Discover/Offer and 655 for Request/ACK. A transaction ID matches messages in an exchange. Those displayed numbers are not a general rule requiring an increment of one; do not use that visual detail as a DHCP algorithm.

Why begin with 0.0.0.0 and a broadcast destination?

The client does not yet have permission to use the offered address and may not know the server's address. It therefore needs special initial addressing. After confirmation, it can use its assigned configuration. The illustrated broadcast exchange is the lecture's scenario, not a claim that every possible DHCP reply is broadcast.

The first two steps can be skipped in certain cases where a client remembers a previously allocated address and requests to reuse it. That does not make Discover and Offer unnecessary for a completely new client.

DHCP can also supply a **subnet mask**, the **first-hop/default router** used to reach other networks, and **DNS information**. DNS, the Domain Name System, helps translate names into IP addresses. Getting an address alone is not the whole configuration job.

Lecture 15 | DHCP encapsulation and allocation

DHCP: example, layer by layer

Source: Lecture 15, PDF pp. 9-11 and 23.

LAN means Local Area Network. An **Ethernet frame** is the unit used on an Ethernet LAN. A **MAC address** identifies a link-layer interface for frame delivery; it is different from an IP address used for network-layer delivery.

Stage	What happens, in order?
1. Client creates request	DHCP constructs the configuration request.
2. UDP wrapper	UDP adds its header, including the client/server port numbers.
3. IP wrapper	IP adds source/destination IP addressing.
4. Ethernet wrapper	Ethernet carries the IP packet. The slide broadcasts the frame to FF:FF:FF:FF:FF:FF, written there as FFFFFFFFFFFF.
5. Server receives	The router containing the DHCP server receives the frame. Each layer identifies the next protocol and passes its payload upward: Ethernet to IP, IP to UDP, UDP to DHCP.
6. Server responds	DHCP prepares the ACK and configuration; UDP, IP and Ethernet wrap the reply for transmission.
7. Client receives	The client unwraps the reply up to DHCP and installs the confirmed settings.

Passing received data to the appropriate higher protocol is **demultiplexing**. Encapsulation is nesting; demultiplexing is deciding which upper handler should receive the unwrapped contents.

Q2: how does a network get its address block?

The organization receives part of its provider ISP's space. The slide's ISP has **200.23.16.0/20** and divides it into eight /23 blocks. A block is a contiguous range of addresses represented by a prefix.

1. Parent host bits = 32 - 20 = **12**; size = 2^12 = **4,096 addresses**.

2. Child host bits = 32 - 23 = **9**; size = 2^9 = **512 addresses**.

3. Number of equal children = 4,096 / 512 = **8**, equivalently 2^(23 - 20).

4. A /23 advances by 2 in the third octet, producing the allocations below.

Organization	Allocated prefix	Organization	Allocated prefix
0	200.23.16.0/23	4	200.23.24.0/23
1	200.23.18.0/23	5	200.23.26.0/23
2	200.23.20.0/23	6	200.23.28.0/23
3	200.23.22.0/23	7	200.23.30.0/23

The final child ends at 200.23.31.255, the end of the parent. Thus the allocation is contiguous, aligned and complete.

Q3: how does an ISP get its block?

The slides describe the IANA/ICANN and regional-registry hierarchy. IANA coordinates address-space allocation to five Regional Internet Registries; those registries and downstream/local registries/providers allocate further. ICANN also has DNS coordination responsibilities. The slide's 2011 exhaustion statement refers to the central IPv4 free pool, not the disappearance of every usable IPv4 address. IPv6's larger address space and NAT are discussed next.

Lecture 15 | Hierarchical addressing

Route aggregation / route summarization

Source: Lecture 15, PDF pp. 12-18.

An **advertisement** in routing is a message telling other routers which destination prefixes a network can reach. **Aggregation** combines several compatible, more-specific prefixes into one less-specific prefix. Instead of making outsiders track every customer separately, an ISP can advertise their common parent.

The original diagram as a story

1. Eight organizations hold the /23 blocks from 200.23.16.0/23 through 200.23.30.0/23.
2. They connect through **Fly-By-Night-ISP**.
3. That ISP advertises **200.23.16.0/20**: send packets for this whole block toward me.
4. The rest of the Internet can use one entry; the ISP's internal routing supplies the more detailed customer paths.
5. Another provider, **ISPs-R-Us**, separately advertises **199.31.0.0/16**.

Why the eight /23 blocks summarize to /20

Eight equals 2^3, so three of the child prefix bits are needed to distinguish the eight children. Remove those three distinctions: **23 - 3 = /20**. Equivalently, 8 x 512 = 4,096 = 2^12 addresses, leaving 32 - 12 = 20 prefix bits. The third octet ranges from 16 = 00010000 to 31 = 00011111: the leading four bits, 0001, stay the same. Along with the first 16 bits, this gives /20.

This shortcut is exact only for contiguous, properly aligned blocks that together fill that aggregate. A shorter common prefix can also include addresses you did not intend to advertise; always check its entire range.

More specific routes: Organization 1 changes provider

The organization keeps **200.23.18.0/23** but connects through ISPs-R-Us. Two possible approaches in the slides are to renumber its devices into the new provider's space, which is costly, or advertise the old prefix separately.

Advertisement	Provider	What it covers
200.23.16.0/20	Fly-By-Night-ISP	The original whole aggregate
199.31.0.0/16	ISPs-R-Us	Its own main allocation
200.23.18.0/23	ISPs-R-Us	Organization 1's more-specific exception

For a packet to **200.23.18.7**, both the /20 and /23 match. The /23 specifies more leading bits, so it wins. A packet to **200.23.20.7** still follows the /20 toward Fly-By-Night. These two destination values are teaching illustrations of the slide's routing situation.

The slides also describe an acquisition/subsidiary variation with the same addressing problem: organizational ownership does not automatically make an old customer prefix fit inside a different provider block.

Key contrast: subnetting divides a block for internal detail; aggregation advertises compatible blocks together. The separately referenced classroom summarization video is not present, so its exact question cannot be reconstructed.

Lecture 15 | Longest prefix matching

Choose the most specific matching entry

Source: Lecture 15, PDF pp. 19-22. Repeated animation pages are combined here.

The router compares a packet's **destination address** against forwarding-table prefixes. A wildcard, written *, means that bit can be either 0 or 1. The match with the greatest number of fixed leading bits wins. A default/otherwise rule is used when no listed specific prefix matches.

Destination pattern from slide	Equivalent prefix	Interface
11001000 00010111 00010*** ********	200.23.16.0/21	0
11001000 00010111 00011000 ********	200.23.24.0/24	1
11001000 00010111 00011*** ********	200.23.24.0/21	2
Otherwise	Default	3

The PDF text splits some wildcard characters over lines. Each third-octet /21 pattern above contains five fixed bits plus three wildcards, completing its eight bits.

Slide example 1

Destination: **11001000 00010111 00010110 10100001**, or **200.23.22.161**.

1. The first 16 bits match the common 200.23 part of the three entries.
2. The third octet is **00010110**. Its first five bits are **00010**.
3. Therefore it matches the interface 0 pattern **00010*****.
4. It does not match the exact third octet **00011000** for interface 1, nor **00011***** for interface 2.
5. The best match is **/21, interface 0**.

Slide example 2

Destination: **11001000 00010111 00011000 10101010**, or **200.23.24.170**.

1. Its third octet is **00011000**, so it matches interface 1's full third octet.
2. Its first five third-octet bits are also **00011**, so interface 2 matches too.
3. Interface 1 fixes 24 leading bits; interface 2 fixes 21 leading bits.
4. Since 24 > 21, choose **interface 1**.

The packet matches two rules, but only the more specific one determines the destination-based forwarding choice. Do not stop at the first matching row unless an implementation explicitly guarantees an ordering that produces the correct longest match.

A practical solving method

Write down the candidate prefix lengths. Eliminate prefixes that disagree with any fixed leading destination bit. Among the remaining matches choose the largest /number. If none match, use the default entry if present; do not invent a path when there is no applicable route.

Why aggregation remains usable: a broad route handles the ordinary case; a longer exception handles a moved organization without requiring the broad route to disappear.

Lecture 15 | MTU and fragmentation

IPv4 fragmentation, reassembly

Source: Lecture 15, PDF pp. 24-28. Image-only details on pp. 25 and 27 were not recoverable.

MTU means Maximum Transmission Unit. In the encapsulation picture here, it is the largest IP packet that a link can carry as the payload of one link-layer frame. It does **not** include the frame's own header/trailer. The wording largest frame on an earlier slide is loose; the later MTU diagram gives the intended layer boundary.

Outer frame area	What it contains	Counted in IP MTU?
Link header	Local-link delivery/control information	No
Frame payload	Complete IP header plus IP payload	Yes
Link trailer	End-of-frame control information, if used	No

A packet fitting one link can be too big for the next. With IPv4, fragmentation can split a datagram into smaller IP datagrams when allowed. Each fragment has its own IP header and a piece of the original payload. Routers forward fragments as independent packets; **the destination reassembles** them into the original datagram.

The story behind the fragmentation diagram

1. A large datagram reaches a router whose outgoing link has a smaller MTU.

2. If fragmentation is permitted, the router divides the payload into pieces sized for that link.

3. Each piece receives an IP header with identifying and positioning information.

4. The fragments travel independently and can arrive out of order.

5. The receiving host uses their offsets and identity information to restore the original payload. Missing fragments prevent complete reassembly.

Header fields that make this possible

Identification: fragments from the same original datagram carry the same ID; reassembly also uses source, destination and protocol information. **MF:** 1 means more fragments follow; 0 means this is the last fragment. MF = 0 alone does not prove the packet was never fragmented: an ordinary unfragmented datagram also has MF = 0 and offset = 0. **Offset:** starting position within the original payload, divided by 8. **DF:** if Don't Fragment is set, a forwarding router must not split the datagram; an oversized packet is dropped and an appropriate error can be reported.

MTU values listed in the lecture

Link type in slide	MTU, bytes	Link type in slide	MTU, bytes
Ethernet	1,500	FDDI	4,352
Token Ring, 4 Mbps	4,464	X.25	576
Token Ring, 16 Mbps	17,914	PPP	296
Wireless	2,312		

These are the lecture's example values for particular link settings. They are not universal values for every device bearing those technology names. Mbps means million bits per second; MTU is measured in bytes, not a transmission rate.

Lecture 15 | Fragmentation numerical

Slide example: 4,000-byte datagram; MTU 1,500

Source: Lecture 15, PDF p. 28 (printed slide footer 4-29). No IP options: each fragment has a 20-byte IP header. The original datagram's 4,000 bytes already include its header.

1. **Find original data.** D = 4,000 - 20 = **3,980 bytes**. This is the information to divide. Do not divide the original IP header as though it were data.

2. **Find data capacity per fragment.** 1,500 - 20 = **1,480 bytes**. Every fragment needs room for its own IP header.

3. **Check the 8-byte rule.** 1,480 / 8 = **185**, an integer. Every non-final fragment's data length must be a multiple of 8 so later offsets can identify its end exactly.

4. **Fill the first two fragments.** They carry 1,480 bytes each, totaling **2,960 data bytes**.

5. **Find the remainder.** 3,980 - 2,960 = **1,020 data bytes**. Add its 20-byte header: the final fragment's Total Length is **1,040 bytes**.

6. **Compute positions.** Fragment 1 starts at data byte 0: offset 0/8 = 0. Fragment 2 starts at data byte 1,480: offset 1,480/8 = 185. Fragment 3 starts at byte 2,960: offset 2,960/8 = 370.

7. **Set MF.** The first two have more data after them, so MF = 1. The third is the last, so MF = 0. All retain the original identification value x.

No.	Original data-byte range (zero based)	Data	Total Length	ID	MF	Offset
1	0-1,479	1,480	1,500	x	1	0
2	1,480-2,959	1,480	1,500	x	1	185
3	2,960-3,979	1,020	1,040	x	0	370

Checks: payloads sum to 3,980. Each total length is at most 1,500. Together the fragments carry 4,040 IP bytes: the original 4,000 plus two extra 20-byte headers. Link-layer framing overhead is not included in this comparison. Offsets count data bytes in the original payload, not headers or fragment numbers.

Lecture 15 - all calculation rules

Rule	Meaning
Children of /p split into /q = 2^(q-p)	Equal-sized aligned subblocks; q >= p.
Aggregate of 2^b equal /q blocks: / (q-b)	Valid only for an exact, contiguous and correctly aligned aggregate.
Longest-prefix choice = matching entry with largest p	More fixed leading destination bits means greater specificity.
D = L - H	Original payload bytes = total datagram bytes minus header.
F = 8 x floor((M-H)/8)	Maximum non-final fragment data, rounded down to a multiple of 8. M = MTU; H = assumed common header bytes.
n = ceiling(D/F)	Fragment count for a fragmented datagram with that fixed header and data capacity.
Last data = D - (n-1)F	Remaining bytes in final fragment.
Fragment total = fragment data + H	Total must not exceed MTU.
Offset = original payload starting byte / 8	For equal full chunks, fragment i has offset (i-1)F/8.
Extra IP header bytes = (n-1)H	Compared with the original single datagram, under the fixed-header assumption.

floor means round down; ceiling means round up. If the original datagram already fits the MTU, do not fragment it. Header options can make header treatment more complex; the supplied numerical assumes none.

Lecture 16 | 13 October 2026

NAT: network address translation

Source: Lecture 16, PDF pp. 3-8.

A local network can use private IPv4 addresses while sharing one public-facing address for outside communication. The boundary router rewrites packet addressing and remembers the translations. **LAN** is the local network; **WAN** refers to the outside/wide-area side in the table.

Strictly, translating addresses alone is Basic NAT. Translating addresses **and transport port numbers** is NAPT, commonly called PAT (Port Address Translation). The lecture's one-public-address example uses this latter mechanism while calling it NAT.

The slide's outgoing-and-returning packet, step by step

The internal host is **10.0.0.1**, the router's local address is **10.0.0.4**, its public address is **138.76.29.7**, and the outside web server is **128.119.40.186:80**. The notation IP:port identifies an address and a transport port together.

Stage	Source endpoint	Destination endpoint	What changes?
1. Host sends	10.0.0.1:3345	128.119.40.186:80	The host starts an outgoing exchange.
2. Router sends outward	138.76.29.7:5001	128.119.40.186:80	NAT changes the source IP and port and records the mapping.
3. Server replies	128.119.40.186:80	138.76.29.7:5001	The server addresses the public endpoint it saw.
4. Router sends inward	128.119.40.186:80	10.0.0.1:3345	NAT looks up the incoming destination and restores the private destination endpoint.

WAN-side mapping	LAN-side mapping
138.76.29.7:5001	10.0.0.1:3345

The table is necessary because multiple internal devices can share the public IP. Their translated port mappings distinguish the conversations. The exact translated port, 5001 here, is selected by the NAT; there is no formula saying to add a constant to 3345. Header checksums affected by translation must be updated.

Advantages and architectural concerns

The slides list public-address conservation, freedom to renumber the private network without notifying outsiders, and the ability to change provider while retaining internal addresses. Private hosts are also not directly globally addressable through ordinary routing. That property is useful, but NAT should not be equated with a complete firewall or a security guarantee.

NAT is controversial because a router modifies transport-layer port information, adds connection-related state inside the network and complicates direct end-to-end reachability. IPv6 addresses the address-space shortage more directly. NAT nevertheless remains widely used in homes, organizations and cellular networks.

Historical correction: the slide says NAT was introduced with Windows 2000. NAT as a general networking technique predates that: RFC 1631 was published in May 1994. Do not memorize the platform statement as the origin of NAT.

Lecture 16 | NAT traversal and IPv6

NAT traversal problem and solutions

Source: Lecture 16, PDF p. 8 names these solutions but does not explain them in detail. The following is introductory teaching context, not an unseen slide reconstruction.

If an outside client wants to initiate contact with an internal server, knowing only the NAT's public address may not be enough. Without an existing or configured mapping, the router does not know which private endpoint should receive the incoming traffic. This is the **NAT traversal** problem.

Solution named in lecture	How its story works
Static configuration / port forwarding	An administrator creates a known mapping from a public port to a chosen internal server endpoint. Incoming traffic matching that mapping can be sent inward.
UPnP / IGD	Universal Plug and Play / Internet Gateway Device mechanisms allow a local application to ask a compatible gateway for a mapping. Availability and permission depend on the gateway's configuration.
Relaying	Both sides communicate through a reachable external relay. The private-side application first makes an outgoing connection; the relay carries data between the parties. This adds an intermediary.

IPv6: motivation

Source: Lecture 16, PDF pp. 9-12.

IPv4 uses 32-bit addresses. IPv6 uses **128-bit addresses**, giving 2^128 possible bit patterns, approximately 3.4 x 10^38. This is 2^96 times as many patterns as IPv4. A larger address space is the main motivation. Other motivations in the slides are a simpler fixed-size base header for processing and fields that identify related traffic for differentiated treatment.

The IPv6 base header is always **40 bytes**. It is longer than the minimum IPv4 header, but its fixed structure removes the variable IHL calculation. Optional information can be carried in extension headers after it.

IPv6 field	Width	What it means
Version	4 bits	Identifies version 6.
Traffic Class	8 bits	Indicates traffic classification/priority information; does not itself promise delivery quality.
Flow Label	20 bits	Labels related datagrams as a flow for handling purposes; it does not reserve bandwidth automatically.
Payload Length	16 bits	Counts bytes after the 40-byte base header, including any extension headers.
Next Header	8 bits	Identifies the header immediately following this one.
Hop Limit	8 bits	Limits forwarding hops, corresponding to the role of IPv4 TTL.
Source address	128 bits	Sender's IPv6 address.
Destination address	128 bits	Intended receiver's IPv6 address.

The first six fields total 64 bits; the addresses add 256 bits. Total = 320 bits = **40 bytes**. The field-width explanation is standard IPv6 background; the separate image-only header slide on PDF p. 11 could not be independently inspected.

Lecture 16 | IPv6 header chain

IPv6 datagram format; IPv6 Next Header Field

Source: Lecture 16, PDF pp. 10-12. Precision notes checked against RFC 8200.

What changes from IPv4?

The IPv6 base header has no header checksum and no variable options area. Its base header is fixed at 40 bytes, and extension headers carry additional information. IPv6 routers do **not** fragment packets. The slide's phrase no fragmentation/reassembly is too broad: source-host fragmentation and destination reassembly can use an IPv6 Fragment extension header.

Read Next Header as follow this label

Every IPv6 base header says what comes next. An extension header, when used, also identifies what follows it. Eventually the chain reaches an upper protocol such as TCP or UDP, or indicates that no further header follows.

Position in a teaching example	Next Header says	Meaning
IPv6 base header	0	A Hop-by-Hop Options extension follows.
Hop-by-Hop extension	43	A Routing extension follows.
Routing extension	6	TCP follows this extension.
TCP header and application data	Outside the IPv6 Next Header chain	TCP interprets its own header and payload.

This example illustrates the chain concept, not a packet specified in the lecture. With no extensions and a UDP payload, the base header can directly name UDP, value 17.

Next Header value listed in lecture	Meaning
6	TCP
17	UDP
58	ICMPv6: IPv6 control/error messaging
59	No Next Header
0	Hop-by-Hop Options extension
43	Routing extension

There are 256 possible values in an 8-bit Next Header field; this does not mean all 256 are available or represent transport protocols. Some identify extension headers.

What counts as the IPv6 payload?

Part	Counted in Payload Length?
Fixed 40-byte IPv6 base header	No
Extension headers, if present	Yes
Transport header	Yes
Application data carried after it	Yes

Teaching numerical: an ordinary IPv6 datagram has Payload Length = 1,200 bytes and 16 bytes of extension headers. Total packet length = 40 + 1,200 = **1,240 bytes**. Bytes remaining after those extensions = 1,200 - 16 = **1,184**. That remainder still includes any transport header; it is not automatically all application data.

The slide says extension headers have no maximum size. Read that as freedom from IPv4's 40-byte options-area limit, not unlimited packet space: extension formats, the packet length and path MTU still constrain packets. Ordinary 16-bit Payload Length handling has a special jumbogram exception beyond this lecture's scope.

Lecture 16 | Transition from IPv4 to IPv6

Tunneling and encapsulation

Source: Lecture 16, PDF pp. 13-18.

Not every router can be upgraded on one agreed day, called a flag day. IPv4-only and IPv6-capable parts must coexist. **Tunneling** carries a complete IPv6 packet as the payload of an IPv4 packet across an IPv4 network. The IPv6 packet is inside another packet; it has not been converted into an IPv4 header.

Follow A's packet to F in the slide

Stage	Header(s) travelling there	What happens?
A to B	IPv6: source A, destination F	A sends the original IPv6 datagram toward the tunnel entrance.
B, tunnel entrance	Outer IPv4: source B, destination E; inner IPv6: A to F	B adds an IPv4 wrapper around the original IPv6 packet.
Across C and D	Same outer B-to-E addressing, carrying inner A-to-F packet	IPv4 routers forward toward E using the outer IPv4 header. They need not interpret the inner IPv6 header.
E, tunnel exit	Removes outer IPv4 wrapper	E recovers the IPv6 datagram. Its end-to-end source/destination remain A and F.
E to F	IPv6: source A, destination F	Ordinary IPv6 forwarding continues.

The logical view shows one tunnel from B to E. The physical view includes IPv4 routers C and D inside that tunnel. Outer addresses name the tunnel endpoints; inner addresses name the original communication endpoints. This distinction is the main point of the slide. Repeated B-to-C captions beneath the illustration should not be read as the actual name of every physical hop.

The slides first compare this with Ethernet encapsulation: an Ethernet frame carries an IP packet over one link. Tunneling applies the nesting idea at the network layer, carrying IPv6 inside IPv4 across a multi-hop region.

Dual stack approach

The lecture asks students to study dual stack but provides no separate worked explanation. A **dual-stack** device supports both IPv4 and IPv6, so it can use the version supported along a given communication path. Both families require configuration and operation during transition. Dual stack alone does not make an IPv4-only endpoint speak IPv6; communication still needs a compatible path or another transition mechanism.

IPv6: adoption

The slides quote approximately 40% of Google clients using IPv6 in **2023** and roughly one-third of US government domains being IPv6-capable. These are historical lecture figures, not current measurements verified for October 2026. The lecture explains slow rollout through replacement cost, the existing installed network and workarounds such as NAT. Changing the network layer is like replacing a building's foundation; changing an application is often more like changing its paint.

Teaching size check: if the inner IPv6 packet is 1,240 bytes and an outer IPv4 header is 20 bytes with no options, the tunneled IP packet is **1,260 bytes** before link framing. Added wrappers consume MTU space; a tunnel is not free in header bytes.

Lecture 16 | Generalized forwarding, SDN

Generalized forwarding: match plus action

Source: Lecture 16, PDF pp. 19-26.

Destination-based forwarding asks which output to use for a destination IP prefix. **Generalized forwarding** can inspect several header fields and apply more kinds of actions. A **flow** here is a set of packets selected by specified field values. The device can be described as a **packet switch** because its rules can implement router-like or link-switch-like behavior.

Flow-table component	Meaning
Match	Conditions that the arriving packet must satisfy, such as a destination IP, source address or transport port.
Action	What to do: forward, drop, modify, copy/log where supported, or send the packet to the controller.
Priority	Resolves overlapping matching rules. Do not assume a table's printed order supplies a missing priority.
Counters / statistics	Count matching packets and bytes so behavior can be measured.

Flow table abstraction: read the slide's rules

Match	Action	In plain language
Any source; destination 3.4.*.*	forward(2)	Send matching packets to physical output interface 2.
Source 1.2.*.*; any destination	drop	Discard packets from that source range.
Source 10.1.2.3; any destination	send to controller	Let the control system handle the matched packet/event.

Here * means any value in that position. A packet from 1.2.7.8 to 3.4.5.6 matches the first two rules. The slide does not state their priorities, so a unique outcome cannot be inferred from those conditions alone. Longest destination prefix is not a universal substitute for flow-table priority.

OpenFlow: flow table entries

OpenFlow provides a way to control compatible packet-switch forwarding behavior through flow entries. The slide's layout includes these match categories:

Category	Fields named in slide
Device context	Ingress port: physical input interface where the packet arrived
Link layer	Source/destination MAC; Ethernet Type; VLAN ID; VLAN priority
Network layer	IP source/destination; IP protocol; IP type-of-service field
Transport layer	TCP/UDP source port; TCP/UDP destination port

An Ethernet Type identifies the protocol carried by the frame. A **VLAN**, Virtual LAN, logically separates link-layer networks; its ID names the virtual LAN and its priority marks traffic class. These are context definitions, not a full VLAN lesson.

Count caution: the slide labels the OpenFlow 1.0 list as 11 header entries, but its displayed layout has 12 match inputs when ingress port is included. Follow the actual named fields rather than memorizing the inconsistent count. Match, action and statistics have distinct jobs: recognize, act, measure.

Lecture 16 | OpenFlow examples

Local rules and device abstraction

Source: Lecture 16, PDF pp. 24-26.

Slide example: matched field	Action	Why this behaves that way
Destination IP 51.6.0.8	Forward to port 6	Destination-based IP forwarding. Other fields are wildcarded.
TCP destination port 22	Drop	A firewall rule blocking traffic addressed to the SSH service port. SSH is a secure remote-login protocol.
Source IP 128.119.1.1	Drop	A firewall rule blocking traffic from that host.
Destination MAC 22:A7:23:11:E1:02	Forward to port 3	Link-layer destination forwarding.

A **firewall** permits or blocks traffic according to policy. A **switch** ordinarily forwards frames using link-layer information. NAT matches address/port information and rewrites it. Thus router, switch, firewall and NAT behavior can all be expressed using match and action, although their conditions and actions differ.

OpenFlow example: program a path through three switches

Source: Lecture 16, PDF pp. 27-28. Goal: packets from h5/h6 in 10.3.*.* to h3/h4 in 10.2.*.* must travel through **s3, then s1, then s2**.

Device	Match conditions in the slide	Action
s3	IP source 10.3.*.* AND IP destination 10.2.*.*	forward(3)
s1	Ingress port 1 AND source 10.3.*.* AND destination 10.2.*.*	forward(4)
s2	Ingress port 2 AND destination 10.2.0.3	forward(3), toward h3
s2	Ingress port 2 AND destination 10.2.0.4	forward(4), toward h4

Follow h5 to h3, one event at a time

1. **h5 (10.3.0.5)** sends a packet destined for **h3 (10.2.0.3)**.

2. At **s3**, the source matches 10.3.*.* and destination matches 10.2.*.*. The action sends it through output port 3 toward s1.

3. It enters **s1 on port 1**. All three match conditions hold, so s1 forwards it out port 4 toward s2.

4. It enters **s2 on port 2**. Destination 10.2.0.3 selects output port 3, reaching h3.

5. For a destination of h4 (10.2.0.4), the first two device actions are the same; s2 selects port 4 instead. h6 (10.3.0.6) also matches s3's same source-range rule.

Do not reuse these as reverse-direction rules: the listed matches specify this particular direction and policy. The controller coordinates local tables so their combined actions produce the intended network-wide path.

Generalized forwarding: summary

The network becomes programmable through rules that recognize packets and choose actions. The slides mention historical active-networking ideas and P4 as a later, more general packet-processing programming approach. Neither is needed to calculate the shown path; the flow conditions and port actions are sufficient.

Lecture 16 | Middleboxes and the IP hourglass

Middleboxes; middlebox functions

Source: Lecture 16, PDF pp. 30-35.

A **middlebox** is an intermediary on the data path doing more than an ordinary IP router's standard forwarding. Its defining feature is the extra function, not a particular physical box shape or location.

Example in lecture	What it adds	Locations mentioned
NAT	Rewrites address/port information	Home, cellular and institutional networks
Firewall / IDS	Enforces traffic policy / detects signs of intrusion	Organizations, service providers, ISPs
Load balancer	Distributes requests among multiple servers	Corporate, provider, data-center and mobile networks
Cache	Stores reusable content near demand to reduce repeated retrieval	Service providers, mobile networks and CDNs
Application-specific middlebox	Performs processing tailored to a particular application/service	Provider, institutional and CDN infrastructure

IDS means Intrusion Detection System: it detects suspicious activity; detection is not necessarily the same as blocking. A **data center** contains servers and supporting networks. The lecture's topology places different middlebox functions at several network locations, showing they are widespread rather than confined to one edge router.

Evolution: hardware, software and NFV

Traditional middleboxes often used proprietary hardware: closed designs tied to a vendor. **Whitebox hardware** uses more general equipment with open interfaces. An **API** is a defined way for software components to communicate/control functions. Programmable match/action behavior allows more differentiation through software.

SDN organizes control and configuration logically centrally. **NFV**, Network Functions Virtualization, implements functions such as translation, security or performance enhancement as software on commodity computing/network/storage resources. Some functions can be hosted in a cloud service. SDN concerns control organization; NFV concerns how a network function is implemented/deployed. They are related, but not synonyms.

The IP hourglass (initially and at middle age)

An hourglass has a narrow middle. In this architectural analogy, many applications and transport choices use one common IP service, which in turn works over many link and physical technologies.

Hourglass level	Examples written in slide	Main idea
Many applications	HTTP, SMTP, RTP, QUIC, DASH	Web, mail, real-time media and streaming-related protocols/techniques
Transport choices	TCP, UDP	Different delivery services above IP
Thin waist	IP	Common network-layer interface spanning diverse technologies
Many link technologies	Ethernet, PPP, PDCP, Wi-Fi, Bluetooth	Different local communication mechanisms
Physical media	Copper, radio, fiber	Different ways to carry signals

The common IP waist allowed applications and underlying networks to evolve independently. The middle-aged hourglass adds caches, firewalls and other functions within the network: more processing surrounds that once-simple waist. This is an architectural observation, not a change to the fixed IPv4 or IPv6 field layout.

Lecture 16 | Internet architectural principles

Architectural Principles of the Internet (RFC 1958)

Source: Lecture 16, PDF pp. 36-40.

The lecture presents three beliefs: **the goal is connectivity**; **IP is the common tool**; and much of the **intelligence belongs at the ends** rather than being hidden inside the core network. The network edge means hosts and application infrastructure; the core means the routers and links carrying traffic between them.

The end-end argument

The question is where a function must live to be complete and correct. Some services can be assisted inside the network, yet the endpoints still need enough knowledge to verify the final result. This is not a claim that routers must perform no useful extra work.

Retell the two reliability diagrams

Arrangement	What happens?	What can the endpoints conclude?
End-to-end reliability	The sender and receiver check the entire transfer. The receiver's confirmation concerns arrival at the final endpoint.	Endpoint checking can establish success for the complete path/application operation.
Hop-by-hop reliability	Each neighboring pair may confirm/recover its local transfer before the next hop.	Local success does not by itself prove final application success; an intermediate device or later step can still fail.

Teaching story: imagine sending a file through several routers. Every link might report successful local receipt. The destination still needs to check that the complete intended file reached the correct application. Internal help can improve performance, but it does not eliminate the endpoint's responsibility for a correct final outcome. The lecture attributes the end-to-end argument to Saltzer, Reed and Clark (1981).

Where's the intelligence?

Architecture in slide	Where computing/decision-making is emphasized
20th-century telephone network	Network switches
Internet before about 2005	End hosts at the edge
Internet after about 2005	Programmable network devices plus extensive application infrastructure at the edge

These are broad architectural comparisons, not exact dates when all networks changed. Middleboxes and SDN complicate the simple edge/core separation, while large services still place major computing resources at the edge.

Chapter 4: done - connect the topics

IP datagrams cross networks through forwarding. Addresses and prefixes identify destinations. DHCP supplies a host's configuration. Aggregation reduces advertised routing detail, with longest-prefix matching preserving exceptions. Fragmentation handles an oversized IPv4 packet when permitted. NAT rewrites endpoints at a boundary; IPv6 expands addressing and changes the header. Generalized forwarding broadens matching/actions; middleboxes add functions beyond ordinary forwarding.

The chapter's final question asks who computes forwarding/flow tables. Its answer is the **control plane**, developed further in the next chapter. The data plane applies those decisions to packets.

Lecture 16 | Formula sheet and comparison

All size rules and operational rules used in this lecture

The uploaded lecture mainly teaches processes rather than numerical exercises. The size examples in these notes are explicitly added teaching examples.

Formula / rule	Meaning
IPv6 address space = 2^128	Approximately 3.4 x 10^38 possible address bit patterns.
IPv6 / IPv4 pattern-count ratio = 2^(128-32) = 2^96	Compares address-space sizes, not transmission speed.
IPv6 base header = 320 bits / 8 = 40 bytes	The base-header size is fixed.
Ordinary IPv6 total length = 40 + Payload Length	Payload Length includes extension headers but excludes the base header; special jumbograms require separate handling.
Bytes after extensions = Payload Length - extension bytes	Still includes any transport header.
Tunneled size = outer header + complete inner packet	For the shown IPv6-in-IPv4 tunnel; excludes link framing.
Possible Next Header bit patterns = 2^8 = 256	Possible encodings, not 256 transport protocols.

Process rule	How to remember it
Outgoing NAT/PAT	Replace private source IP:port with mapped public IP:port; remember mapping.
Returning NAT/PAT	Match public destination endpoint to the stored mapping; restore private destination.
IPv4 tunnel wrapper	Outer addresses name tunnel entry and exit; inner addresses name original endpoints.
Flow-table decision	Apply matching conditions, resolve overlaps using specified priorities, then perform action and count traffic.

IPv4 versus IPv6: keep these pairs separate

Property	IPv4	IPv6
Address width	32 bits	128 bits
Base/normal header	20 bytes minimum, up to 60	Fixed 40 bytes
Header-length field	IHL needed for variable header	No IHL in fixed base header
Length field meaning	Total Length includes IP header	Payload Length excludes 40-byte base header
Upper-header identification	Protocol field	Next Header chain
Loop-prevention field	TTL	Hop Limit
IP header checksum	Present	Absent from base header
Optional information	Options inside variable IPv4 header	Extension headers after base header
Router fragmentation	Possible when allowed	Routers do not fragment; sources can use Fragment extension

Final concept checks

Can DHCP and NAT be the same job? No. DHCP assigns configuration; NAT rewrites packet addressing. They may run in the same router.

Does a public NAT IP uniquely identify the internal application? No. The port mapping and associated protocol/state are needed to deliver the return traffic correctly.

Does an IPv4-only router in a tunnel need IPv6 support? It forwards on the outer IPv4 header. The tunnel endpoints must perform the encapsulation/decapsulation.

Does a flow rule necessarily use longest-prefix matching? No. Generalized rules can overlap across multiple fields; their specified priority resolves the policy.

Source audit and remaining gaps

What was covered, and what was not recoverable

Page references throughout these notes use physical PDF page numbers, not printed slide-footer numbers. Repeated animation states are merged; title/attribution and administrative slides are not turned into technical lessons.

Supplied file	Material used	Limits that must remain visible
Lecture 13, 42 pages	Chapter 4 starts at p. 22; technical material through p. 41.	pp. 1-21 are Chapter 3 and intentionally excluded; p. 42 is Chapter 3 administration.
Lecture 14, 22 pages	Addressing/subnetting pp. 3-20.	Image-only content in pp. 6-9, 17-18 was not fully inspectable. Standard explanations are supplied, but exact hidden examples/labels cannot be certified. FLSM/VLSM videos absent. p. 21 only says Midterm 1 solutions were discussed; it supplies no question/solution.
Lecture 15, 30 pages	DHCP through fragmentation, pp. 3-28.	Image-only pp. 7, 25 and 27 may contain unrecovered diagram details. Summarization video absent. p. 30 is a Chapter 3 quiz title without questions.
Lecture 16, 41 pages	Technical content pp. 3-40.	Separate IPv6 header picture on p. 11 not inspectable; standard field explanations supplied. NAT traversal and dual stack are named study tasks with little slide detail; their explanations are labeled teaching context.

The roadmap's **What's inside a router** heading names input/output ports, switching, buffers and scheduling. A switch fabric transfers packets internally; a buffer temporarily stores waiting packets; scheduling chooses which waiting packet goes next. Those are orientation definitions only. The four supplied files do not provide dedicated lessons, diagrams or numerical problems on these mechanisms, so full coverage of those roadmap items cannot be claimed.

Administrative date conflict in the sources

Lecture 15 lists Chapter 4 assignment/quiz dates as 20 October 2026; Lecture 16 lists 22 October 2026. The later slide gives the later date, but the authoritative course announcement should resolve the conflict. No deadline is silently assumed here.

Clarifications against primary standards

These links support limited technical clarifications; they do not replace the course PDFs or supply imagined lecture content.

Reference	Clarification used
RFC 791 - https://www.rfc-editor.org/rfc/rfc791.html	IPv4 field widths; IHL units; fragment offset, DF/MF and reassembly identity
RFC 8200 - https://www.rfc-editor.org/rfc/rfc8200.html	IPv6 header fields, payload length, extension headers and source-only fragmentation
RFC 4632 - https://www.rfc-editor.org/rfc/rfc4632.html	CIDR prefixes, hierarchy and aggregation terminology
RFC 1918 - https://www.rfc-editor.org/rfc/rfc1918.html	Exact private IPv4 ranges
RFC 1122 - https://www.rfc-editor.org/rfc/rfc1122.html	Historical special addresses, limited broadcast and loopback
RFC 3022 - https://www.rfc-editor.org/rfc/rfc3022.html	Basic NAT versus address-and-port translation
RFC 1631 - https://www.rfc-editor.org/rfc/rfc1631.html	NAT documented in May 1994, before Windows 2000

To complete the source record: the original readable PDFs/images for the flagged pages and the three classroom videos are needed. No unseen numerical values, class discussion solutions or animation orders have been asserted as recovered facts.